What is Matplotlib?

Matplotlib is a Python library used primarily for **creating visualizations and plots from data**.

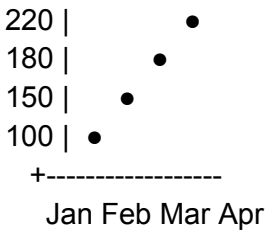
At a high level:

```
      Python
        ↓
NumPy / Pandas
        ↓
    Matplotlib
        ↓
Visual Representation
```

For example, if you have:

Month	Sales
Jan	100
Feb	150
Mar	180
Apr	220

Python can calculate these values, but Matplotlib can turn them into:



MATLAB Appears

Before Matplotlib, one of the major environments for numerical computing and visualization was **MATLAB**.

MATLAB was designed around mathematical and numerical computation.

Its name comes from:

MATrix **LAB**oratory

MATLAB made it relatively easy to do things like:

Numerical computation
+
Matrix operations
+
Visualization

For example, MATLAB users could write relatively simple commands to plot mathematical functions.

This became extremely popular among:

- Engineers
- Scientists
- Researchers
- Universities

NumPy Changes Scientific Python

NumPy provided powerful numerical arrays and mathematical operations.

Instead of working with Python lists alone:

x = [1, 2, 3, 4]

scientists could work with numerical arrays:

import numpy as np

x = np.array([1, 2, 3, 4])

This created a foundation:

Python
↓
NumPy
↓
Scientific Computing

But there was still a major missing piece:

Visualization

Python could calculate the data, but scientists needed an effective way to plot it.

The Birth of Matplotlib

Matplotlib was created by **John D. Hunter**.

This happened in the early 2000s.

Hunter was working in the field of **neurobiology** and needed a way to visualize scientific data using Python.

Instead of relying on MATLAB, he wanted a Python-based plotting solution.

That led to the development of Matplotlib.

The project began around **2003**.

Why Not Just Use MATLAB?

This is an important question.

If MATLAB already had excellent visualization, why create Matplotlib?

Because Python offered several advantages.

Python

Open source
+
General-purpose language
+
Extensible
+
Scientific ecosystem

Researchers wanted to combine:

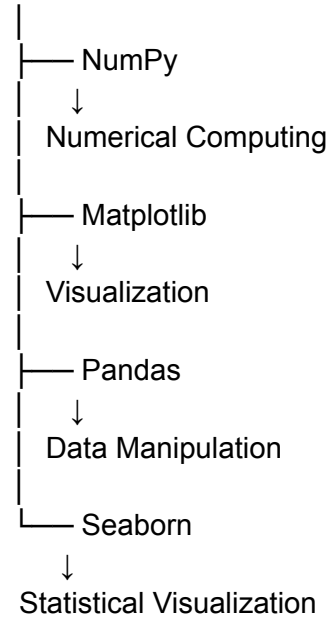
Python
+
Numerical computation
+
Visualization
+

Scientific applications

instead of switching between different environments.

Evolution of the Python Visualization Ecosystem

The ecosystem gradually developed into something like:



This is why we should learn:

NumPy → Pandas → Matplotlib → Seaborn

rather than treating each library as completely unrelated.

Matplotlib's Architecture Evolved

One of the most important things you will learn later is that Matplotlib isn't simply:

plt.plot()

There is a deeper architecture.

Conceptually:

Your Python Code
↓
Pyplot / Matplotlib API
↓
Figure
↓
Axes
↓
Artists
↓
Renderer
↓
Backend
↓
Screen / PNG / PDF / SVG

Distribution Plots

What is a Distribution?

A **distribution** tells us how the values of a variable are spread across different ranges.

Suppose we have the ages of 20 people:

```
ages = [
  21, 22, 23, 24, 25,
  25, 26, 27, 28, 29,
  30, 30, 31, 32, 33,
  34, 35, 36, 40, 50
]
```

Instead of looking at 20 individual numbers, we want to understand:

- Where are most values concentrated?
- What is the typical value?
- How spread out are they?
- Are there extreme values?
- Is the data symmetric?
- Is it skewed?

- Are there multiple groups?

That's **distribution analysis**.

Why Distribution Matters in AI/ML

Suppose you're building a salary prediction model.

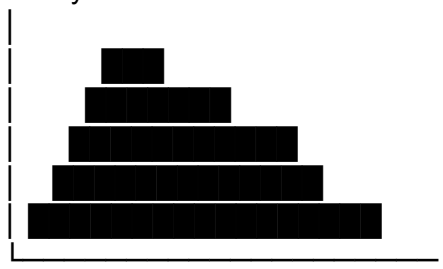
Your dataset contains:

Age
Salary
Experience
Education
Department

Before training the model, you need to understand the features.

For example:

Salary



You might discover:

> Most salaries are between ₹30K–₹80K, but a small number are above ₹2 lakh.

That matters because extreme values can affect:

- Mean
- Standard deviation
- Regression models
- Scaling
- Outlier detection
- Feature engineering

So distribution plots aren't just for visualization.

They help you **understand the data-generating behavior**.

The Main Distribution Plots

For your Matplotlib + Seaborn :

1. Histogram
   ↓
2. KDE Plot
   ↓
3. Histogram + KDE
   ↓
4. Box Plot
   ↓
5. Violin Plot
   ↓
6. ECDF
   ↓
7. Rug Plot
   ↓
8. Q-Q Plot

The first five are particularly important.

Histogram

A histogram groups numerical values into **bins**.

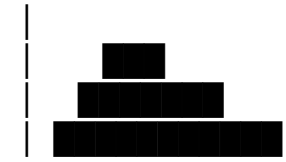
Example:

```
import matplotlib.pyplot as plt

ages = [21, 22, 23, 24, 25, 25, 26, 27,
      28, 29, 30, 30, 31, 32, 33, 34,
      35, 36, 40, 50]

plt.hist(ages)
plt.show()
```

Conceptually:



Age

The X-axis represents:

Value ranges

The Y-axis represents:

Number of observations

What is a Bin?

This is a **critical concept**.

Suppose ages range from 20 to 50.

We can divide them into:

20–25
25–30
30–35
35–40
40–45
45–50

These intervals are called **bins**.

For example:

Age	Bin
21	20–25
24	20–25
27	25–30
29	25–30
33	30–35

Histogram:

20–25 → ■■■■■
25–30 → ■■■■■■■
30–35 → ■■■■■■■■■
35–40 → ■■■■
40–45 → ■■
45–50 → ■

Matplotlib Histogram

Basic:

plt.hist(data)

Important parameters:

```
plt.hist(
  data,
  bins=20,
  edgecolor="black"
)
```

Learn these parameters:

- `bins`
- `range`
- `density`
- `alpha`
- `histtype`
- `orientation`

For example:

```
plt.hist(
  ages,
  bins=10,
  density=True
)
```

`density=True` changes the histogram from raw counts toward a **probability density representation**.

Seaborn Histogram

Seaborn makes distribution visualization easier.

import seaborn as sns

sns.histplot(data=ages)

You can specify:

```
sns.histplot(
  data=ages,
  bins=10
)
```

For a DataFrame:

```
sns.histplot(
  data=df,
  x="salary"
)
```

This is something you'll use constantly during EDA.

Histogram with Multiple Categories

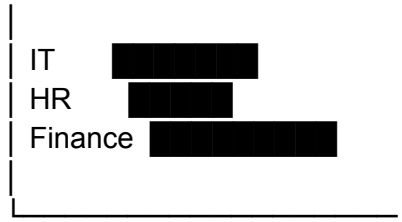
Suppose:

Salary
Department

You want to compare salary distributions.

```
sns.histplot(
  data=df,
  x="salary",
  hue="department"
)
```

Conceptually:



Now you're doing **comparative distribution analysis**.

KDE — Kernel Density Estimate

KDE stands for:

Kernel Density Estimate

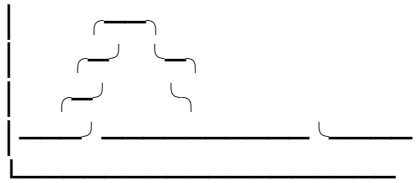
It estimates the underlying probability density of the data.

Instead of:

Histogram

KDE gives a smooth curve:

Density



Why KDE?

A histogram depends heavily on bin boundaries.

For example:

Bin A | Bin B | Bin C

Changing the bins can significantly change the appearance.

KDE tries to provide a smoother representation of the underlying distribution.

Seaborn:

sns.kdeplot(data=ages)

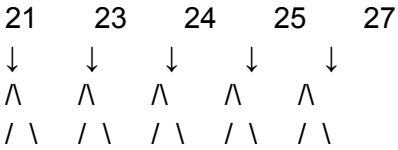
How KDE Conceptually Works

Suppose we have:

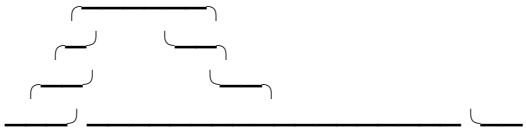
21 23 24 25 27

KDE essentially places a small smooth kernel around each observation.

Conceptually:



Then these individual contributions are combined:



This produces the smooth density curve.

KDE Bandwidth

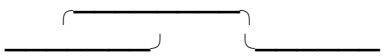
One advanced KDE concept you must understand is **bandwidth**.

Bandwidth controls the smoothness of the KDE.

Very small bandwidth

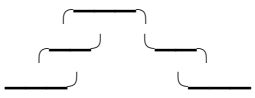
Too sensitive to individual observations.

Very large bandwidth



Too smooth.

Appropriate bandwidth



Therefore:

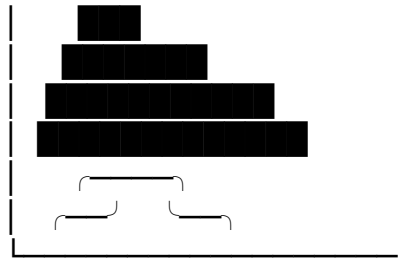
> **Bandwidth controls how much smoothing is applied to the estimated density.**

Histogram + KDE

One of the most useful EDA plots:

```
sns.histplot(
  data=df,
  x="salary",
  kde=True
)
```

You get:



This combines:

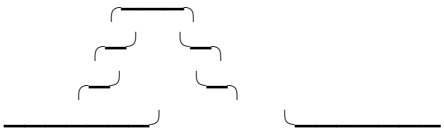
Histogram → actual observed frequency

KDE → estimated smooth density

Normal Distribution

A very important distribution in statistics and ML.

It looks approximately like:



Characteristics:

- Symmetric
- Mean ≈ Median ≈ Mode
- Bell-shaped

Example:

Human height
Measurement errors

Some biological measurements

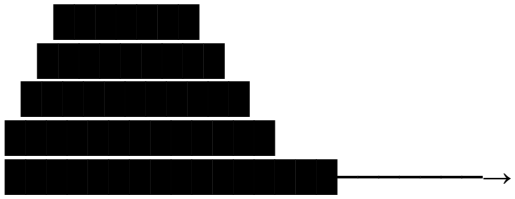
But remember:

> **Not every real-world feature is normally distributed.**

Right-Skewed Distribution

Example: income.

Most people may have moderate incomes, while a small number have extremely high incomes.



This is called:

> **Positive skew / Right skew**

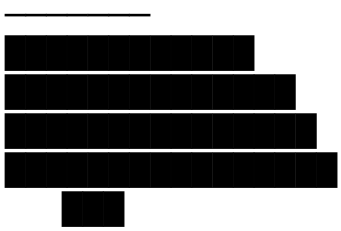
Typically:

Mean > Median

This is very important in real-world datasets.

Left-Skewed Distribution

Opposite situation:



The tail extends toward the left.

Called:

> **Negative skew / Left skew**

Typically:

Mean < Median

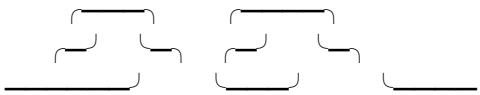
Bimodal Distribution

Suppose you analyze customer age.

Maybe there are two major customer groups:

Students
+
Working professionals

The distribution might look like:



Two peaks = **bimodal distribution**.

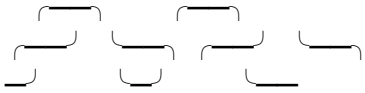
This is an extremely useful discovery.

It can indicate:

> Your data may contain multiple underlying populations.

Multimodal Distribution

More than two peaks:



This may indicate multiple groups or processes generating the data.

Uniform Distribution

Values are approximately evenly distributed:

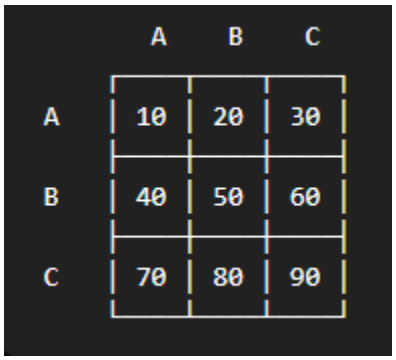
[redacted]

For example, a properly generated random number between 0 and 1.

What is a Matrix Plot?

A **matrix plot** is a visualization where data is represented in a **row × column matrix**, and each cell contains a value.

Example:



Instead of displaying only the numbers, we can use colors:

This is generally called a **heatmap**.

Why Do We Need Matrix Plots?

Suppose you have 10 features.

A correlation matrix contains:

10×10=100

values.

Looking at:

df.corr()

may produce a large table.

It is difficult to identify patterns quickly.

A heatmap turns that into:

High correlation → strong color

Low correlation → lighter color

Negative correlation → opposite color

So the purpose is:

> **Convert a numerical matrix into a visual pattern that makes relationships easier to identify.**

Most Important Use Cases

For interviews, remember these.

1. Correlation matrix

Most common in EDA.

Feature A ↔ Feature B

Feature A ↔ Feature C

Feature B ↔ Feature C

---

2. Confusion matrix

Very important in classification.

	Predicted	
	0	1

Actual 0	TN	FP
Actual 1	FN	TP

---

3. Missing-value matrix

Visualize where data is missing.

	Age	Salary	City
1	✓	✓	✓
2	✓	✗	✓
3	✗	✓	✓

---

4. Feature relationship matrix

Useful during EDA.

---

5. Similarity matrix

Used in:

- Recommendation systems
- NLP
- Clustering
- Embeddings
- Search systems

---

6. Distance matrix

Used in:

- Clustering
- Nearest-neighbor analysis

- Geospatial analysis
- ML algorithms

Matrix Plot vs Heatmap

This is an important interview distinction.

Matrix plot is a broad concept.

Heatmap is a specific visualization technique for displaying matrix values using color.

Think:

Matrix
↓
Numerical values
↓
Color encoding
↓
Heatmap

So if an interviewer asks:

> "How do you visualize a correlation matrix?"

A strong answer is:

> "I calculate the correlation matrix using Pandas and visualize it using a Seaborn heatmap."

```
lmplot()
regression plot is used in machine learning so we will not fully discuss what, how and where it is used but we will see how it is created and what are the values we can put inside it
```

Plotly is python library which is used to design graphs , especially interactive graphs , whereas cufflinks connect plotly with pandas to create graphs and charts of dataframes directly.